

# A Comparative Study of Deep Reinforcement Learning Algorithms: DQN, PPO, SAC, and TD3

Camille Challier<sup>1</sup>, Céline Kalbermatten<sup>2</sup>, Wesley Monteith-Finas<sup>1</sup>, Luca Mouchel<sup>2</sup>, Elia Mounier-Poulat<sup>2</sup>, Colin Smyth<sup>3</sup>  
<sup>1</sup>Neuro-X; <sup>2</sup>Data Science; <sup>3</sup>Computer Science;

## 1 Introduction

From navigating robots in unpredictable environments to playing complex games, Deep Reinforcement Learning (RL) drives intelligent decision-making in challenging settings. RL agents learn by interacting with their environment, observing states, taking actions, and receiving rewards to guide future decisions. In this project, we re-implemented and compared four leading Deep RL algorithms, DQN, PPO, SAC, and TD3, across several classic OpenAI Gym environments.

## 2 Methods

### Deep RL Algorithms

**Deep Q-Network (DQN)**<sup>1</sup> is a value-based, off-policy method for discrete action spaces. It approximates the Q-function  $Q(s, a; \theta)$  with a neural network and updates parameters by minimizing:

$$L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} \left[ (y_i - Q(s, a; \theta_i))^2 \right]$$

where  $y_i = \mathbb{E}_{s' \sim \mathcal{E}} [r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \mid s, a]$  is the target for iteration  $i$  and  $\rho(s, a)$  is a probability distribution over sequences  $s$  and actions  $a$ , referred to as the behavior distribution.

**Soft Actor-Critic (SAC)**<sup>3</sup> is an off-policy algorithm for continuous control that uses two Q-networks and adds an entropy bonus to encourage exploration. It minimizes the soft Bellman residual:

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[ \frac{1}{2} (Q_\theta(s_t, a_t) - \hat{Q}(s_t, a_t))^2 \right]$$

where the target  $\hat{Q}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V_{\hat{V}}(s_{t+1})]$  includes a slowly updated value network.

**Proximal Policy Optimization (PPO)**<sup>2</sup> is an on-policy algorithm for both discrete and continuous tasks that directly optimizes a stochastic policy while limiting large updates. The clipped surrogate objective is:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$

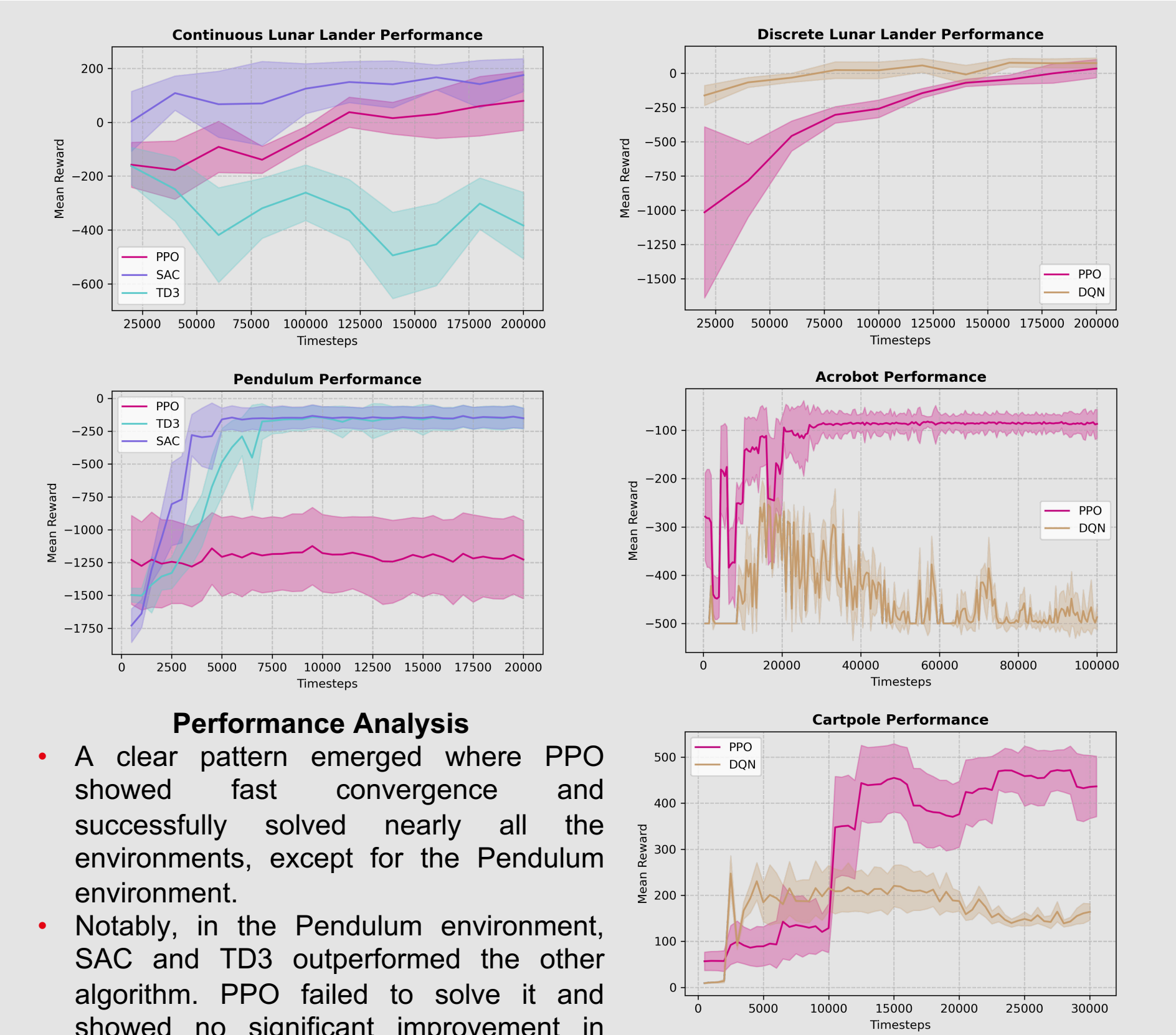
where  $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$  is the probability ratio,  $\hat{A}_t$  is the estimated advantage, and  $\epsilon$  is a hyperparameter, typically set to 0.2.

**Twin Delayed Deep Deterministic Policy Gradient (TD3)**<sup>4</sup> is an off-policy actor-critic method for continuous actions that addresses overestimation and instability in Deep Deterministic Policy Gradient (DDPG) and also uses two Q-networks. The target for the Q-function update is:

$$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_{\phi}(s'))$$

where  $\theta'_i$  are the parameters of the two target Q-networks, and  $\pi_{\phi}$  is a slowly updated target actor network.

## 3 Results



### Performance Analysis

- A clear pattern emerged where PPO showed fast convergence and successfully solved nearly all the environments, except for the Pendulum environment.
- Notably, in the Pendulum environment, SAC and TD3 outperformed the other algorithm. PPO failed to solve it and showed no significant improvement in performance over time steps.

| Environment             | Algorithm | FPS             | SF           | PD          |
|-------------------------|-----------|-----------------|--------------|-------------|
| Acrobot                 | DQN       | 1444.1 ± 554.9  | 99.2 ± 1.4   | 96.8 ± 1.4  |
|                         | PPO       | 1036.5 ± 134.5  | 140.9 ± 0.7  | 67.3 ± 0.3  |
|                         | SAC       | -               | -            | -           |
|                         | TD3       | -               | -            | -           |
| Cartpole                | DQN       | 3923.8 ± 110.1  | 98.4 ± 0.0   | 93.7 ± 0.0  |
|                         | PPO       | 2810.9 ± 67.7   | 138.8 ± 0.0  | 66.0 ± 0.0  |
|                         | SAC       | -               | -            | -           |
|                         | TD3       | -               | -            | -           |
| Discrete Lunar Lander   | DQN       | 1487.2 ± 87.8   | 104.9 ± 0.0  | 95.3 ± 0.0  |
|                         | PPO       | 1716.1 ± 107.6  | 146.7 ± 0.0  | 66.8 ± 0.0  |
|                         | SAC       | -               | -            | -           |
|                         | TD3       | -               | -            | -           |
| Continuous Lunar Lander | DQN       | -               | -            | -           |
|                         | PPO       | 1449.0 ± 198.1  | 146.7 ± 0.1  | 65.9 ± 0.0  |
|                         | SAC       | 327.9 ± 451.0   | 306.7 ± 1.4  | 114.6 ± 0.1 |
|                         | TD3       | 1161.1 ± 1422.5 | 589.6 ± 0.9  | 127.1 ± 0.0 |
| Pendulum                | DQN       | -               | -            | -           |
|                         | PPO       | 2141.9 ± 6.6    | 137.0 ± 1.0  | 65.4 ± 0.5  |
|                         | SAC       | 250.2 ± 3.9     | 2937.7 ± 1.2 | 114.6 ± 0.0 |
|                         | TD3       | 315.0 ± 4.4     | 5781.6 ± 1.2 | 127.1 ± 0.0 |

Table 1: Comparison of mean and standard deviation of training speed in frames per second (FPS), storage footprint (SF) in kilobytes and parameter density (PD) in parameters per kilobyte for RL algorithms across different environments.

- DQN tends to be the most computationally expensive in discrete environments, where PPO trains significantly faster.
- SAC seems to be the most time efficient to train.
- PPO stores policies in a compact and efficient format, with high parameter density (~66–67 params/kB).

- In contrast, SAC and TD3 generate large model files (up to ~5800 kB), resulting in lower storage efficiency and greater memory usage.
- SAC and TD3 scale effectively to continuous control environments like Pendulum, where they outperform PPO.
- DQN is not designed for continuous action spaces and is therefore unsuitable for such tasks.

## Gymnasium Environments<sup>5</sup>



**Acrobot:** Swing a two-link pendulum to reach a target height, with discrete actions in a continuous state space.

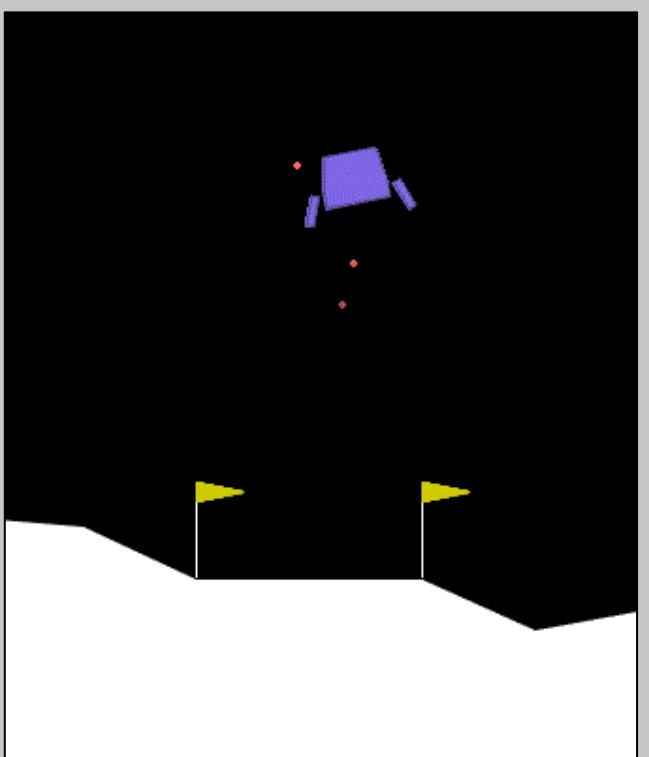


**Cartpole:** Balance a pole on a moving cart using left/right forces, a simple control task with a discrete action space.



**Pendulum:** Apply torque to swing a pendulum upright, a continuous control problem.

**Lunar Lander:** Control a spacecraft's thrusters for a soft lunar landing, testing both precision and stability in discrete and continuous settings.



## 4 Discussion & Conclusion

- PPO shows fast convergence and better performance than DQN on Cartpole and Acrobot.
- In Pendulum and Continuous Lunar Lander, PPO fails to reach optimal performance, while SAC and TD3 clearly outperform it.
- SAC and TD3 are better suited for complex continuous control, though they require more computational resources.
- Overall, PPO remains the simplest and most efficient algorithm for fast prototyping and baseline performance.

## References

- V. Mnih, K. Kavukcuoglu, D. Silver et al., "Playing atari with deep reinforcement learning," 2013.
- J. Schulman, F. Wolski, P. Dhariwal et al., "Proximal policy optimization algorithms," 2017.
- T. Haarnoja, A. Zhou, P. Abbeel et al., "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," 2018.
- S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor critic methods," 2018.
- G. Brockman, V. Cheung, L. Pettersson et al., "Openai gym," arXiv preprint arXiv:1606.01540.